

15 Graph Theory and Network



15.1 Undirected Graphs

Edges have no direction — connections are symmetric; a symmetric adjacency matrix describes an undirected graph, and its powers count paths through intermediate nodes

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

In an undirected graph, edges have no direction — connections are symmetric

2

The adjacency matrix of an undirected graph is a symmetric matrix

3

Powers of the adjacency matrix count paths through multiple intermediate nodes

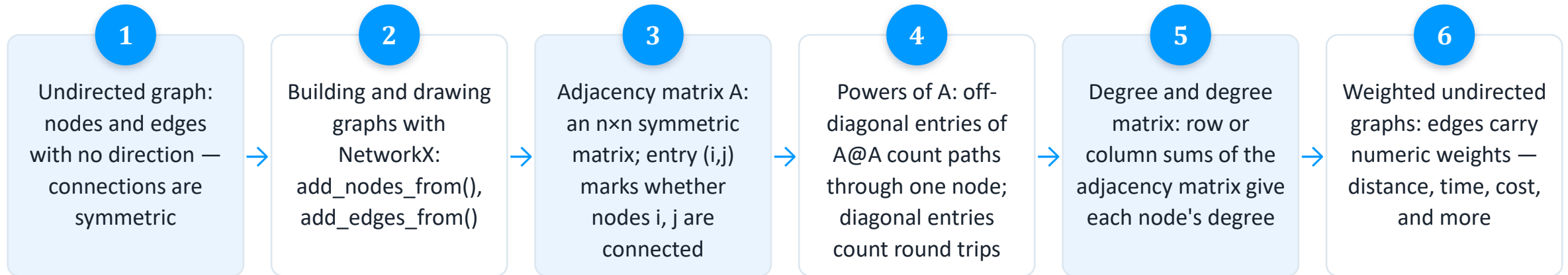
4

Row and column sums of the adjacency matrix give each node's degree

5

Weighted undirected graphs: edges carry numeric weights

From undirected graphs to adjacency matrices, path counts, degree, and weight

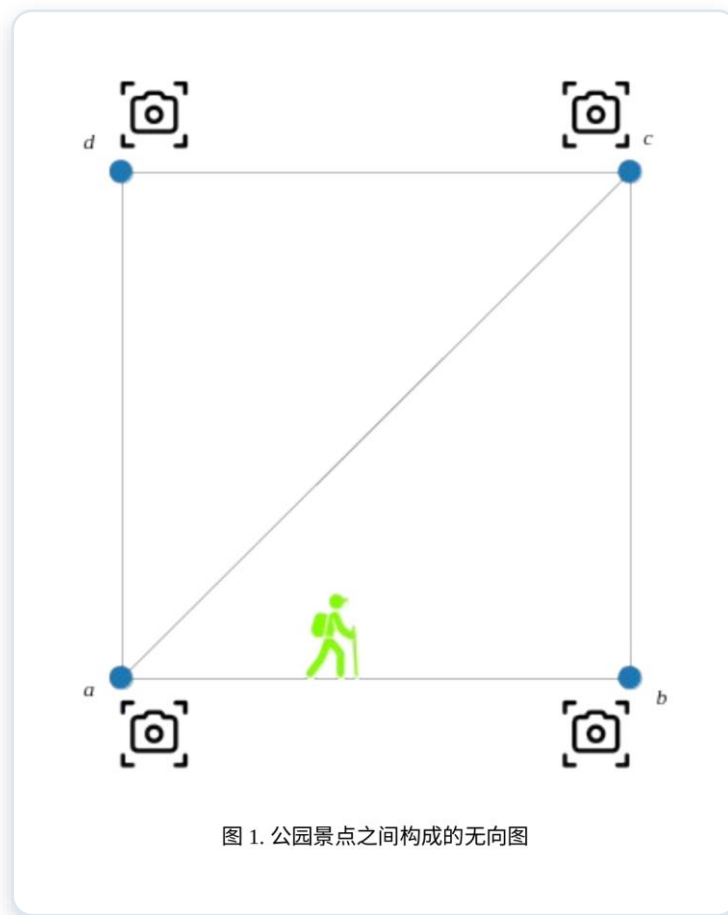


Matrices are the perfect tool for describing and analyzing graphs and networks

- ① Graph theory studies structures built from nodes and edges; graphs and networks are everywhere — in machine learning, AI, and everyday life
- ② This chapter is an introduction to graph theory and networks; the linear algebra tools from earlier chapters show up throughout
- ③ 15.1 Undirected Graphs: edges have no direction, and the adjacency matrix is symmetric (this section)
- ④ 15.2 Directed Graphs: edges have direction, so the adjacency matrix is no longer symmetric — read alongside this section
- ⑤ 15.3 Transition Matrices: probability matrices describing random walks and state transitions on a graph
- ⑥ 15.4 The Laplacian Matrix and Spectral Clustering: combining eigendecomposition with graph structure for clustering

Figure 1 · An undirected graph formed by park attractions

- The park's 4 attractions become 4 nodes a, b, c, d; hiking trails between them become edges — 5 in all: ab, ac, ad, bc, cd, none with a direction



Building and drawing an undirected graph with NetworkX

Code 1. Drawing an undirected graph with NetworkX

- Import the graph library NetworkX (as nx); nx.Graph() creates an empty undirected graph object
- add_nodes_from(['a','b','c','d']) and add_edges_from([...]) batch-add nodes and edges
- In an undirected graph, ('a','b') and ('b','a') refer to the same edge — order doesn't matter
- A dictionary pos manually sets each node's 2D coordinates; nx.draw_networkx() draws nodes, edges, and labels in one call

```
## 初始化
import matplotlib.pyplot as plt
a import networkx as nx

## 无向图
b undirected_G = nx.Graph()

## 添加节点
c undirected_G.add_nodes_from(['a', 'b', 'c', 'd'])

## 添加边
d undirected_G.add_edges_from([('a', 'b'),
                                ('a', 'c'),
                                ('a', 'd'),
                                ('b', 'c'),
                                ('c', 'd')])

## 定义节点坐标
e pos = {'a': [0, 0],
         'b': [1, 0],
         'c': [1, 1],
         'd': [0, 1]}

## 可视化
f plt.figure(figsize = (6,6))
nx.draw_networkx(undirected_G, pos = pos, node_size = 188)
```



An undirected graph can be represented by a symmetric adjacency matrix

$$A = [0 \ 1 \ 1 \ 1; 1 \ 0 \ 1 \ 0; 1 \ 1 \ 0 \ 1; 1 \ 0 \ 1 \ 0]$$

- For an undirected graph with n nodes, the adjacency matrix is an $n \times n$ symmetric matrix A ; the entry in row i , column j marks whether nodes i, j are connected (1 if connected, 0 if not)
- Since edges have no direction, A is symmetric about its main diagonal: entry (i,j) equals entry (j,i)
- A 's diagonal entries are all 0, since no node in Figure 1 has an edge to itself; such an edge, if it existed, would be called a self-loop

Figure 2 · Every entry of the undirected graph's adjacency matrix

- Checking each entry against the edges: a and b are connected, so row 1/col 2 and row 2/col 1 are both 1; c and d are not connected, so those entries are 0

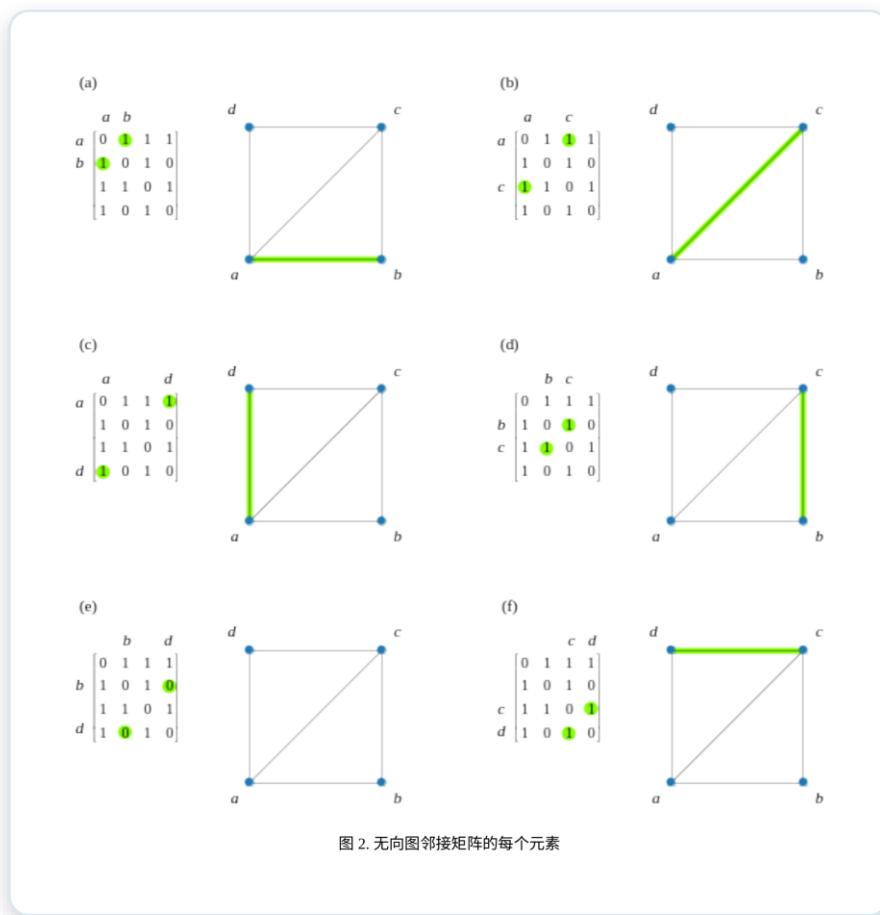


Table 1 · Several undirected graphs on 4 nodes and their adjacency matrices (top)

- The same 4 nodes a, b, c, d can form many different undirected graphs, each with its own symmetric adjacency matrix — work through each pair below

表 1. 4 个节点构造的几种无向图及邻接矩阵，表格示例来自《数据有道》

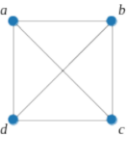
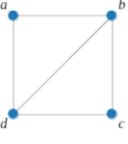
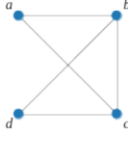
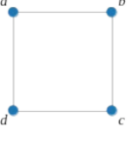
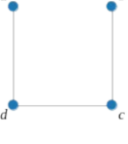
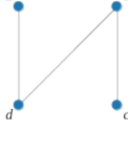
无向图	邻接矩阵	无向图	邻接矩阵
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$

Table 1 · Several undirected graphs on 4 nodes and their adjacency matrices (continued)

- Continue analyzing the remaining rows; compare how the pattern of 1s in the matrix shifts as the connections change

	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$

Building an undirected graph from an adjacency matrix

Code 2. From adjacency matrix to undirected graph

- Define a symmetric 4×4 adjacency matrix directly with `numpy.array()`
- `nx.Graph(adjacency_matrix, nodetype=int)` automatically adds an edge wherever the matrix has a 1
- A dictionary mapping = `0:'a', 1:'b', 2:'c', 3:'d'` renames the default integer node labels to letters
- `nx.relabel_nodes()` applies the renaming, making the graph easier to read and label

```
## 初始化
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

## 定义邻接矩阵
a adjacency_matrix = np.array([[0, 1, 1, 1],
                               [1, 0, 1, 0],
                               [1, 1, 0, 1],
                               [1, 0, 1, 0]])

## 创建无向图
# 用邻接矩阵创建无向图
b undirected_G = nx.Graph(adjacency_matrix, nodetype=int)

# 构造节点映射
c mapping = {0: 'a', 1: 'b', 2: 'c', 3: 'd'}

# 重命名节点
d undirected_G = nx.relabel_nodes(undirected_G, mapping)

## 定义节点坐标
pos = {'a': [0, 0],
       'b': [1, 0],
       'c': [1, 1],
       'd': [0, 1]}

## 可视化
plt.figure(figsize = (6,6))
nx.draw_networkx(undirected_G, pos = pos,
                  node_size = 188, with_labels=True)
```

Figure 3 · a reaches three nodes directly — row 1 of A

- The 1s in row 1 of A (excluding the diagonal) mark which nodes a can reach directly, with no intermediate stop: b, c, d

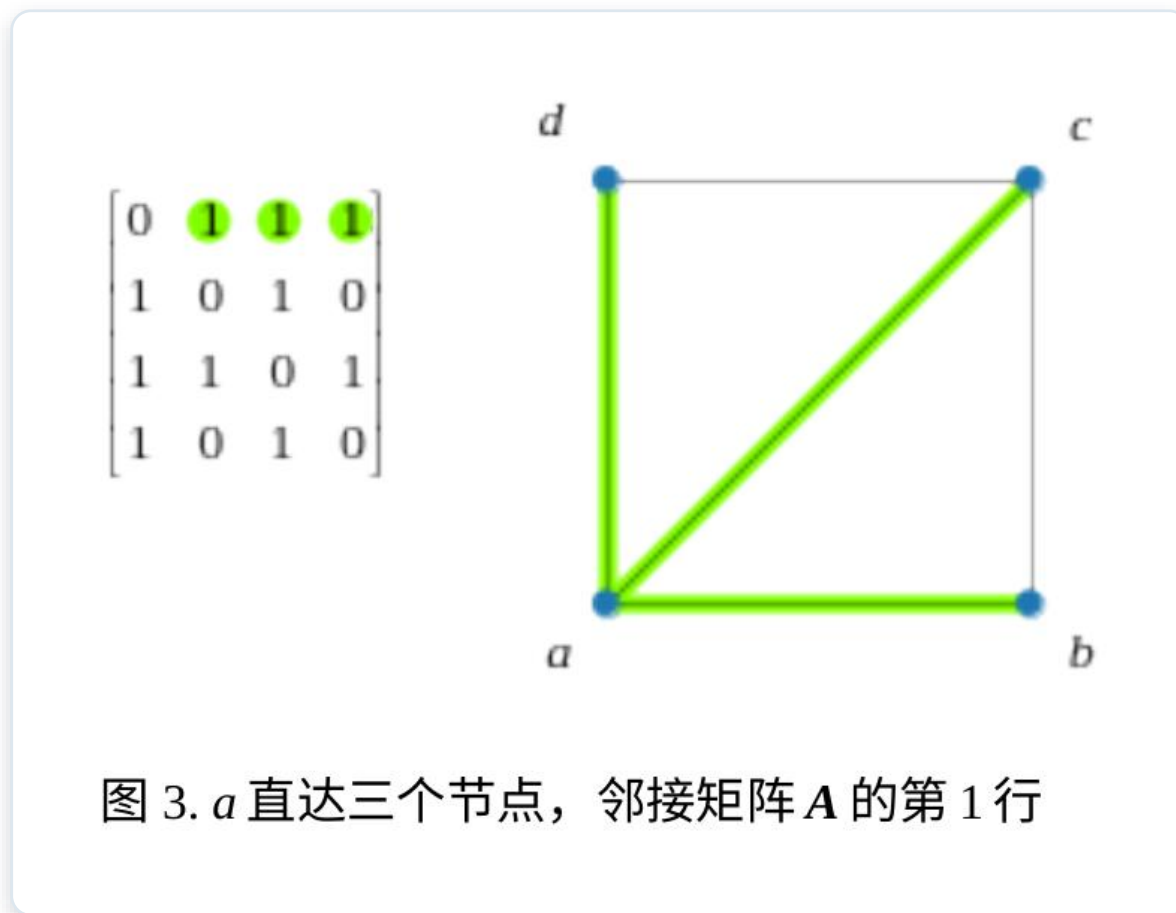


Figure 4 · a reaches three nodes directly — column 1 of A

- Column 1 of A carries the same information; since A is symmetric, rows and columns can be read interchangeably

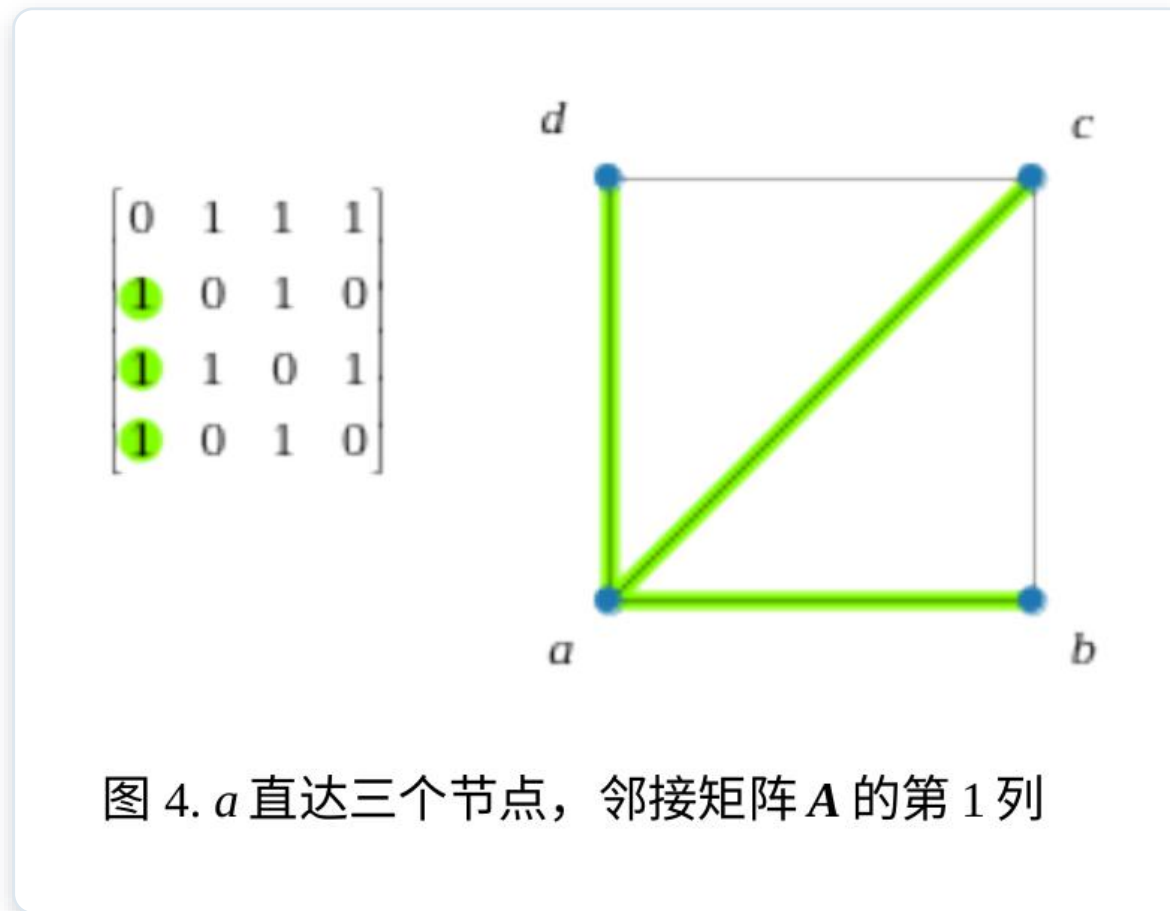


Figure 5 · Paths from a to c through one intermediate node

- Entry (1,3) of A^2 (or equivalently (3,1)) is 2 — the two paths from a to c through one node: $a \rightarrow b \rightarrow c$ and $a \rightarrow d \rightarrow c$

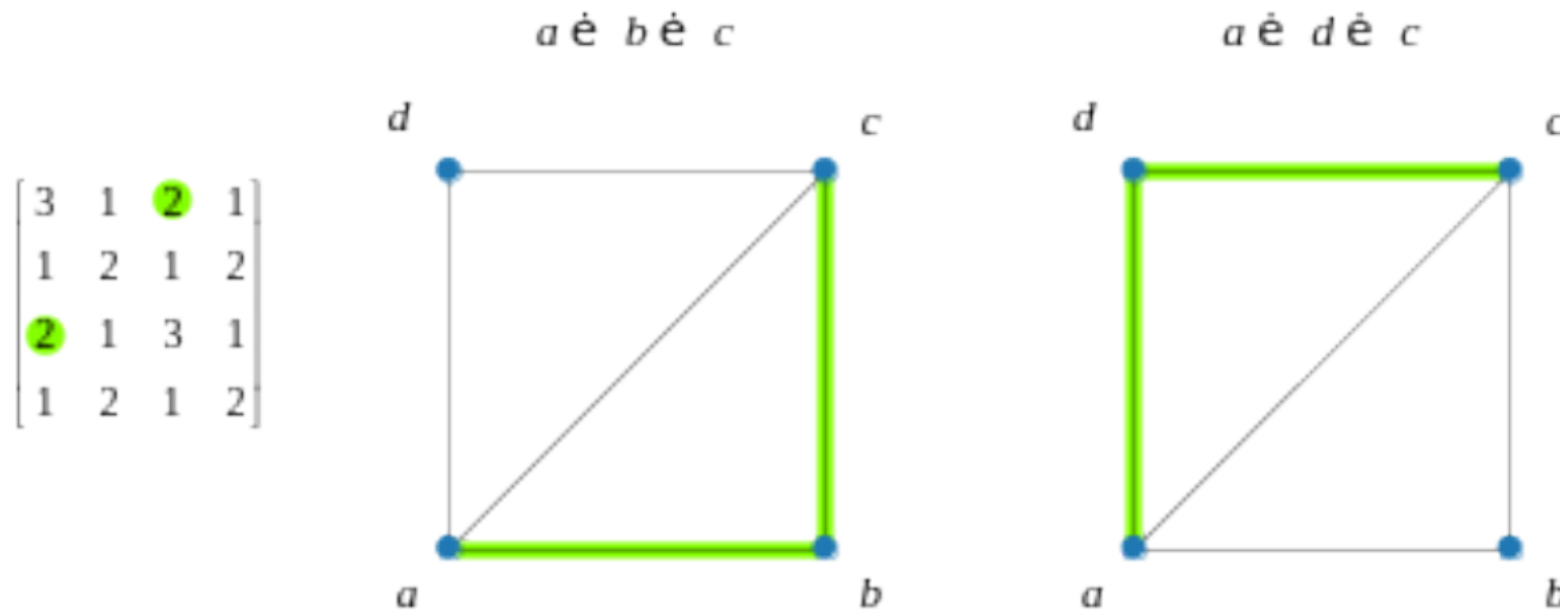


图 5. 节点 a 途径一个节点到达 c 的路径

Figure 6 · Paths from a back to a through one intermediate node

- The diagonal entries of A^2A count round trips through one node: there are 3 paths that leave a , pass through one node, and return to a

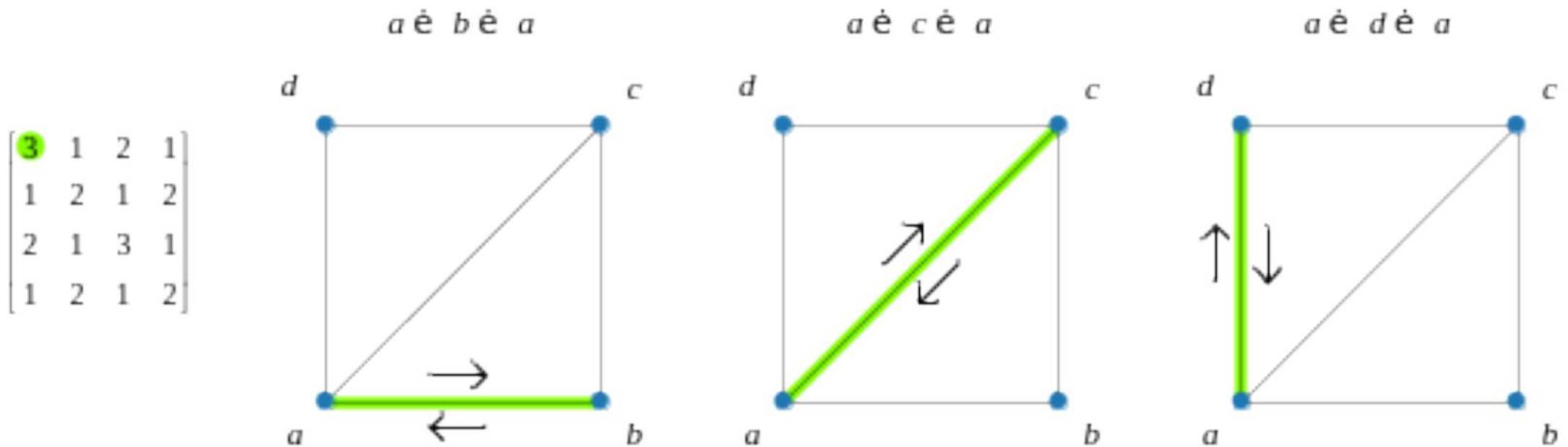


图 6. 节点 a 途径一个节点再回到 a 的路径

Row or column sums of the adjacency matrix give each node's degree

① Degree: the number of edges connecting a node to the rest of the graph — a measure of how active that node is

② Degree matrix D : a diagonal matrix recording every node's degree; every off-diagonal entry is 0

③ Column sum: $\mathbf{1}^T \mathbf{A}$ gives every node's degree, then $\text{diag}()$ builds the degree matrix D

④ Row sum: since an undirected graph's adjacency matrix A is symmetric, $\mathbf{A}\mathbf{1}$ gives the same result as the column sum

Figure 7 · The degree matrix of an undirected graph

- Figure 1's degree matrix is $D = \text{diag}(3, 2, 3, 2)$ — nodes a, b, c, d have degrees 3, 2, 3, 2; in a social network, high degree is like being a “social butterfly,” low degree suggests a preference for solitude

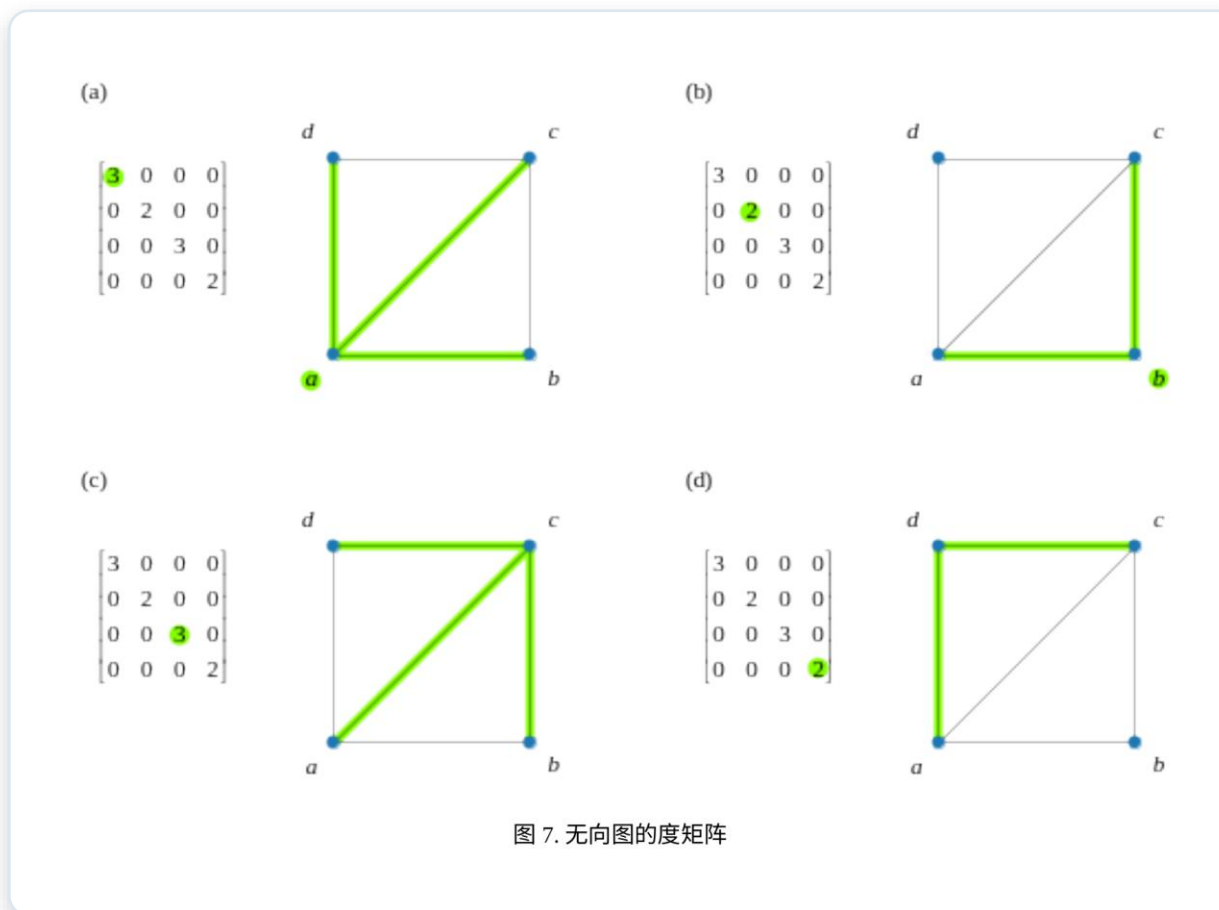
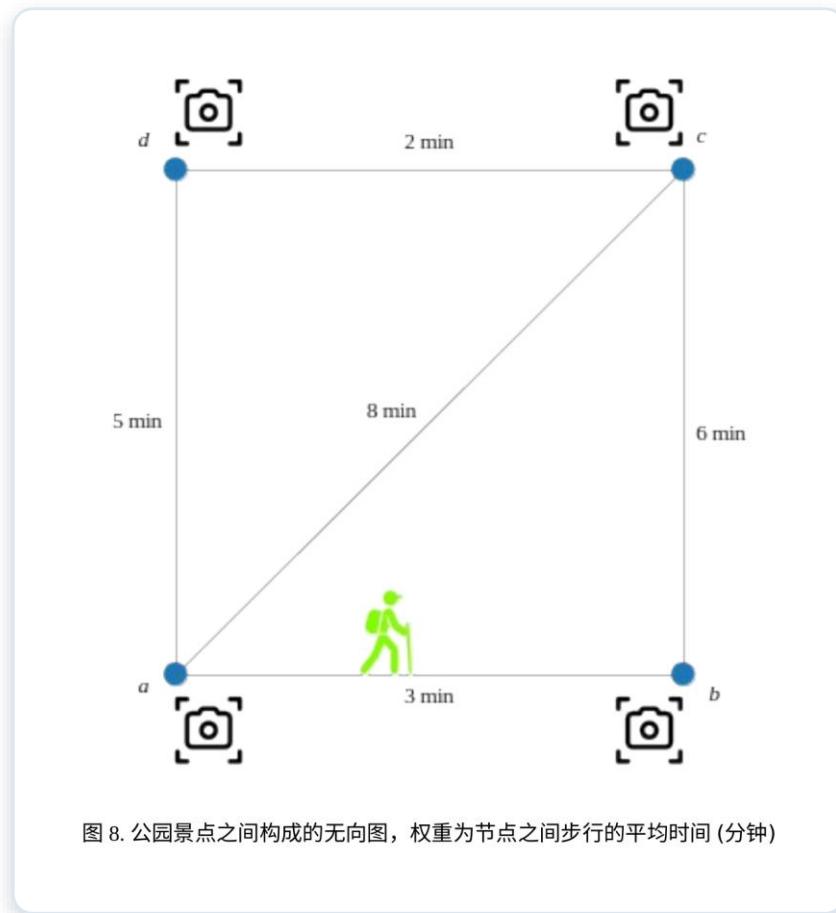


Figure 8 · The park graph, weighted by walking time

- Every edge of a weighted undirected graph carries a numeric weight — here, the average walking time (minutes) between attractions; weight could just as easily represent distance or cost



SUMMARY

Symmetric graph, symmetric matrix, degree hidden inside a sum

① In an undirected graph, edges have no direction — connections are symmetric

③ The k -th power of $\mathbf{A}$ has entries counting paths between nodes through $k-1$ intermediate nodes

⑤ Summing the adjacency matrix by column (or row) gives each node's degree

② The adjacency matrix $\mathbf{A}$ is symmetric; without self-loops its diagonal is 0, and each entry marks whether two nodes are connected

④ Degree is the number of edges touching a node; the degree matrix $\mathbf{D}$ is a diagonal matrix of all node degrees

⑥ A weighted undirected graph's edges carry numeric weights — distance, time, cost, and more

Key Terms in This Section

Undirected Graph *A graph whose edges have no direction; connections are symmetric*

Self-loop *An edge connecting a node to itself*

Degree Matrix *A diagonal matrix recording every node's degree*

Adjacency Matrix *An $n \times n$ symmetric matrix whose entries mark whether nodes are connected*

Degree *The number of edges connecting a node to the rest of the graph*

Weighted Undirected Graph *An undirected graph whose every edge carries a numeric weight*

FURTHER READING

Further Reading

Review · Chapter 3, Block Matrix Multiplication

The computational logic behind the adjacency matrix power A^k

Review · Chapter 2, Matrix Transpose

The symmetric-matrix identity $A^T = A$ underlies the undirected-graph adjacency matrix

Preview · 15.2 Directed Graphs

Once edges have direction, the adjacency matrix is no longer symmetric — read alongside this section

Open-source Code Repository

Code and figures for the entire book: <https://github.com/Visualize-ML>

Exercises Q1 – Q9

Q1 Use NetworkX to build and draw every undirected graph in Table 1.

Q3 Compute the adjacency matrix of every graph in Table 1.

Q5 Learn the concepts of a complete graph and an empty graph, and identify which entries in Table 1 are which.

Q7 In Python, verify that the sum of the degrees of nodes a, b, c, d in Figure 1 equals exactly twice the number of edges.

Q9 Use `numpy.trace()` to compute the trace of $A@A$ for the Figure 1 adjacency matrix, and explain what this number means in graph terms.

Q2 Compute the degree of every node in every graph in Table 1.

Q4 Compute the degree matrix of every graph in Table 1.

Q6 In Python, compute $A@A@A$ for the Figure 1 adjacency matrix A , and explain what its diagonal and off-diagonal entries represent.

Q8 Design your own 5-node undirected graph, draw it with NetworkX, and compute its adjacency matrix and degree matrix.

Exercises Q10 – Q17

Q10 In Python, verify that summing the undirected adjacency matrix A by column gives exactly the same result as summing it by row.

Q12 In Python, compute $A+A@A$ for the graph you designed in Q8, and analyze what its diagonal and off-diagonal entries mean.

Q14 Use NetworkX's `nx.degree()` to compute each node's degree in Figure 1 directly, and compare it with your hand calculation.

Q16 Explain what other features a weighted undirected graph's weights could represent besides walking time.

Q11 Explain why an adjacency matrix's diagonal entries are all 0, unless the graph has a self-loop.

Q13 Give real-world examples of situations that can be modeled as undirected graphs, and draw the corresponding graphs.

Q15 In Python, build the weighted version of Figure 1 (Figure 8), and label the weights with `nx.draw_networkx_edge_labels()`.

Q17 Write a Python function that checks whether a given square matrix is a valid undirected-graph adjacency matrix (symmetric, zero diagonal).

Exercises Q18 – Q25

Q18 For the graph you designed in Q8, compute the number of paths between any two nodes using at most 2 intermediate nodes.

Q20 In Python, compare the degree matrices of a complete graph (every pair of nodes in Table 1 connected) and an empty graph (no edges).

Q22 Explain the practical meaning of node degree in social network analysis, with an example of what a highest-degree node represents.

Q24 Based on this section, explain why matrices and matrix decompositions are important tools for analyzing graphs and networks.

Q19 Explain why entries of A 's k -th power tend to grow larger as k increases.

Q21 Using Matplotlib, plot the heatmap of the adjacency matrix for the graph you designed in Q8, and compare it against the graph's structure.

Q23 In Python, verify that every node in an n -node complete graph has degree $n-1$.

Q25 Preview Section 15.2 and guess what properties a directed graph's adjacency matrix might have differently, with your reasoning.

10 Prompts to Explore with AI

1 Explain the essential difference between undirected and directed graphs, and how their adjacency matrices differ.

2 Explain why an undirected graph's adjacency matrix must be symmetric.

3 Explain what the powers A^2 and A^3 of an adjacency matrix mean in graph terms.

4 Explain the definitions of degree and degree matrix, and their role in graph algorithms.

5 Give examples of real-world networks that can be modeled as undirected graphs.

10 Prompts to Explore with AI

6

Explain what a self-loop is, and how it changes the adjacency matrix.

7

Explain the difference between weighted and unweighted undirected graphs, and what a weight can represent.

8

Explain why summing the adjacency matrix by either row or column gives a node's degree.

9

Explain the concepts of complete graph and empty graph, and what their adjacency matrices look like.

10

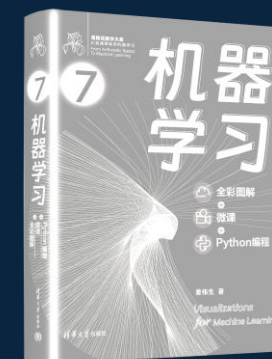
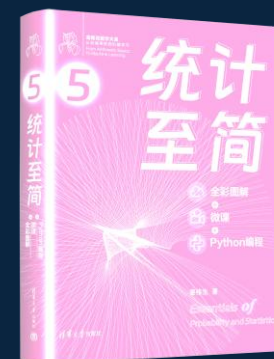
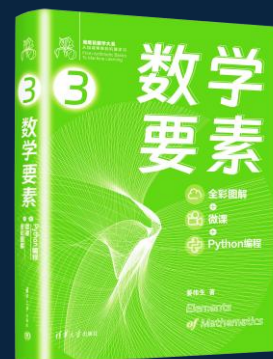
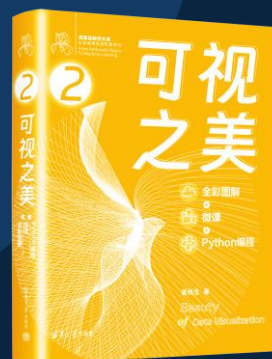
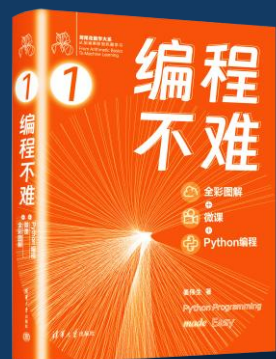
Explain the roles the degree matrix and adjacency matrix each play in computing the Laplacian matrix.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

